

FAMILY OS

Architecture & Product Vision

Version 1.0 • March 2026

The operating system for family life

Contents

1. Product Vision & Problem Statement
2. System Architecture
3. Modular Feature System
4. Module Specifications
5. AI Agent Architecture
6. Voice API & External Access (Future Backend)
7. Interactive Walkthrough System
8. Data Model
9. Integration Strategy
10. Development Phases & Roadmap

1. Product Vision & Problem Statement

The Problem

Families operate on a patchwork of disconnected tools: a shared Google Calendar that nobody updates consistently, a group text that moves too fast to track, a ClassDojo app the teacher uses, a school portal for assignments, a budgeting spreadsheet nobody opened since January, and a fridge whiteboard with chores that nobody erases. The result is that the operational load of running a family falls disproportionately on one parent who tries to keep everything in their head.

Family OS exists to replace the mental load with a system. Not another app to check, but a unified operating system that actively works for the family — pulling information from the sources that already exist (email, school platforms, calendars), organizing it automatically, and surfacing the right information to the right person at the right time.

Core Principles

- **Pull, don't push.** Family OS should require minimal manual data entry. It connects to the family's existing data sources and extracts information automatically.
- **Agent-first, not dashboard-first.** The AI agent is not a feature — it is the product. Every module should be able to be populated, queried, and managed through the agent.
- **Modular by design.** Every family is different. Features are opt-in modules that can be enabled, disabled, and configured independently.
- **Multi-surface access.** Family OS answers questions wherever you ask them — in the app, through Alexa, through Siri, via a daily email digest. The app is one interface, not the only interface.
- **Privacy-first.** Family data is sensitive. Raw email content is never stored. Classification results and extracted data are stored locally. The architecture supports self-hosting.

Target User

The primary user is a parent (often a mom) who is the de facto “chief operating officer” of the household. They manage school communications, coordinate schedules, track spending, assign chores, and generally keep the family running. Family OS gives them a co-pilot.

Secondary users are the other parent (who wants visibility without the operational overhead) and the children (who interact with chores, homework, and goals assigned to them).

2. System Architecture

Current Architecture (Client-Side)

The app is currently a fully client-side React + Vite + TypeScript application with no backend. All state lives in React state hooks in App.tsx and is persisted to localStorage. The AI agent (onboarding, email classification) calls the Anthropic API directly from the browser. Gmail access is handled via the Anthropic API's MCP server passthrough.

Technology Stack

Layer	Technology	Purpose
Framework	React 19 + TypeScript	UI components and type safety
Build Tool	Vite 6	Dev server, bundling, env vars
Styling	Tailwind CSS (CDN)	Utility-first CSS, Notion-inspired design
Routing	React Router (HashRouter)	Client-side page navigation
Charts	Recharts	Data visualization on Dashboard/Insights
AI — Reasoning	Anthropic API (Claude Sonnet)	Onboarding agent, email classification, insights
AI — Vision/OCR	Google Gemini API	Receipt scanning, document OCR
Email Access	Gmail MCP via Anthropic API	Read school emails, ClassDojo notifications
Persistence	localStorage	All app state persisted per-key

Data Flow

All data flows through a centralized state model in App.tsx. External data enters through three pipelines:

- **Onboarding Agent:** Conversational chat → Anthropic API → structured family profile → generates seed data for all modules.
- **Email Intelligence:** Gmail MCP → raw emails → Anthropic classification → router creates Calendar events, Assignments, Action Items.
- **Manual Entry:** User fills forms/modals → state updates → localStorage persistence.

Future Architecture (With Backend)

When ready to add a backend (required for voice API, scheduled jobs, and multi-device sync), the recommended path is:

Component	Technology	Rationale
-----------	------------	-----------

API Server	Node.js + Express or Hono	Same language as frontend, lightweight
Database	SQLite via Turso (or Supabase/Postgres)	Simple, serverless-friendly, real SQL
Auth	Clerk or Supabase Auth	Family accounts with parent/child roles
Hosting	Vercel (frontend) + Railway/Fly.io (API)	Easy deploys, good free tiers
Scheduled Jobs	Vercel Cron or Railway Cron	Email scanning, daily digests, heartbeats
Voice API	Express routes + Alexa Skills Kit / Siri Shortcuts	REST endpoints for voice assistants

KEY ARCHITECTURE DECISION

The client-side architecture can continue indefinitely for a single-family, single-device use case. The backend becomes necessary when any of these are needed: (1) voice assistant access, (2) multi-device sync, (3) scheduled background jobs (cron/heartbeats), (4) shared access without localStorage. Design all modules to be backend-agnostic — they should work with localStorage today and swap to API calls tomorrow.

3. Modular Feature System

Design Philosophy

Every feature in Family OS is a self-contained module that can be independently enabled or disabled. This serves three purposes: (1) families only see what they need, reducing cognitive overhead; (2) onboarding can progressively enable modules based on what the family cares about; (3) the codebase stays clean with clear boundaries between features.

Module Registry

The app maintains a central module registry that controls which features are active. This registry is configured during onboarding and editable in Settings.

Module ID	Display Name	Default	Dependencies
schoolwork	Schoolwork Tracker	ON	None
chores	Chore Manager	ON	None
calendar	Family Calendar	ON	None
finance	Family Finances	OFF	None
email_intel	Email Intelligence	OFF	Gmail connection
goals	Goals	OFF	None
habits	Habit Tracker	OFF	None
todos	Shared To-Do Lists	ON	None
insights	Family Insights	ON	2+ modules active
heartbeats	Heartbeats & Automations	OFF	Backend (future)
voice_api	Voice Assistant Access	OFF	Backend (future)

Module Interface

Each module implements a standard interface that the app shell uses to register, render, and manage it:

Property	Type	Description
id	string	Unique module identifier (e.g., 'schoolwork')
name	string	Human-readable display name
icon	Lucidelcon	Icon component for sidebar and navigation
route	string	URL route (e.g., '/schoolwork')

component	React.FC	The page component to render
defaultEnabled	boolean	Whether enabled by default for new families
dependencies	string[]	Module IDs that must be active for this module
stateSlice	string	localStorage key for this module's data
dashboardWidget	React.FC null	Optional widget for the Dashboard page
settingsPanel	React.FC null	Optional configuration panel for Settings

How Modules Affect the App

- **Sidebar Navigation:** Only shows links for enabled modules. Disabled modules are completely hidden.
- **Dashboard:** Renders dashboardWidget components only for enabled modules. The Dashboard is a composition of active module widgets.
- **Settings:** Shows configuration panels for all modules (including disabled ones), with a toggle to enable/disable each. Disabled modules show a description and an enable button.
- **Onboarding:** The AI agent asks which areas matter to the family and enables the corresponding modules. It can also recommend modules based on the family's profile (e.g., if kids are school-age, recommend schoolwork).
- **Insights:** Only analyzes data from enabled modules. The Family Flow Report adapts to what's active.
- **Routes:** Disabled module routes redirect to Dashboard. No 404s from disabled features.

4. Module Specifications

4.1 Core Modules (Existing)

Schoolwork Tracker

Tracks assignments, tests, projects, and reading logs per child. Supports manual entry and automatic import via the Email Intelligence pipeline. Each assignment has a subject, due date, estimated time, status, and optional teacher name.

Future enhancements: grade tracking, GPA calculation, study time logging, teacher contact directory.

Chore Manager

Assigns and tracks household chores per family member. Supports one-time and recurring (weekly) frequencies. Chores have due dates, assignees, and completion status.

Future enhancements: chore rotation schedules, allowance integration (connect to Finance module), streak tracking (connect to Habits module), age-appropriate chore suggestions.

Family Calendar

Centralized calendar showing events from all sources: manual entries, email-imported events, and (future) Google Calendar sync. Events have titles, start/end times, locations, and source provenance.

Future enhancements: Google Calendar bidirectional sync, conflict detection, travel time estimation, recurring event support.

Family Finances

Tracks transactions, budget categories, and savings goals. Supports manual entry and (future) bank feed integration. Budget categories have spending limits with visual progress tracking.

Future enhancements: bank feed integration, receipt OCR (via Gemini), allowance management linked to chores, recurring bills, spending trend analysis.

Family Insights

AI-powered analysis page that cross-references data from all active modules. Currently uses a static “Family Flow Report” — the architecture supports dynamic generation via the Anthropic API with full family context.

The Insights module becomes more powerful as more modules are enabled. With schoolwork + chores + calendar active, it can identify patterns like “schoolwork completion drops on heavy calendar days.” With habits + goals, it can track progress toward family objectives.

4.2 New Modules

Shared To-Do Lists

A simple, shared task list for the family. Unlike chores (which are recurring responsibilities) and assignments (which are school-specific), to-dos are ad-hoc tasks that anyone can add: “pick up milk,” “RSVP for birthday party,” “call the dentist.”

To-dos have: title, assignee (optional), due date (optional), category/list (e.g., “Groceries,” “Household,” “Kids”), priority, and completion status. Families can create multiple named lists.

Integration points: the Email Intelligence agent can auto-create to-dos from action items in school emails. The voice API can add to-dos via Alexa/Siri. Future integration with Todoist, Apple Reminders, or Google Tasks via MCP.

Goals

Individual and family-level goal setting. A goal has a title, description, owner (individual or “family”), type (savings, behavioral, academic, custom), target metric, current progress, and deadline.

Goal types and connections:

- **Savings goals:** Link to the Finance module. Progress auto-updates as transactions are logged against the goal category.
- **Behavioral goals:** Link to the Habits module. Example: “Family reads together 5x/week” — progress tracks via habit check-ins.
- **Academic goals:** Link to the Schoolwork module. Example: “Maya maintains a B+ average in math” — progress tracks via grade entries.
- **Custom goals:** Manual progress tracking. Example: “Run a 5K together by June.”

Habit Tracker

Daily/weekly habit tracking for individuals and the family as a whole. A habit has: title, frequency (daily, specific days, weekly), assignee, streak count, and completion history.

Family habit examples: eat dinner together, no screens after 8pm, family game night. Individual habit examples: 30 minutes of reading, practice piano, exercise.

Visualization: streak calendars (GitHub-style contribution graph), weekly completion percentages, trend lines. The Insights module can analyze habit consistency alongside other data.

Heartbeats & Automations (Future — Requires Backend)

Automated, scheduled actions that run without user intervention. Inspired by OpenClaw’s heartbeat pattern. Each heartbeat is a simple rule: trigger (schedule or event) → action (notification, scan, digest).

Pre-built heartbeat templates:

- **Morning Briefing:** Every day at 7am — summarizes today's calendar, pending homework, open to-dos, and habit check-ins due.
- **Evening Wrap-up:** Every day at 8pm — asks if chores are done, homework is finished, habits are checked in.
- **Email Scan:** Every 2 hours during school days — checks Gmail for new school communications.
- **Weekly Digest:** Sunday at 9am — summarizes the week's spending, habit streaks, completed vs. missed tasks.
- **Deadline Alert:** 24 hours before any assignment due date or event — sends a reminder.

Setup should be dead simple: toggle a heartbeat on/off, optionally customize the time. No cron syntax, no code. The Settings page shows a list of available heartbeats with toggles and schedule pickers.

5. AI Agent Architecture

Agent Roles

The AI layer consists of several specialized agents, all powered by the Anthropic API (Claude Sonnet), each with a distinct system prompt and tool access:

Agent	Trigger	Tools / MCP	Output
Onboarding Agent	First-time setup	None (pure conversation)	Family profile, module config, email config
Email Scanner	Manual button or scheduled	Gmail MCP	Raw email content
Email Classifier	After email scan	None (reasoning only)	Classified emails with extracted data
Insights Agent	Insights page load or weekly	None (reasoning with context)	Family Flow Report, observations, recommendations
Voice Query Agent	Alexa/Siri API request (future)	App data API	Spoken response to natural language query
Walkthrough Agent	Post-onboarding	None (generates step configs)	Interactive tour step sequence

Shared Context Pattern

All agents receive a “family context” block in their system prompt that includes: family name, parent name, children (names, ages, grades, schools), enabled modules, and relevant data summaries. This ensures consistent, personalized responses regardless of which agent is active.

The family context is assembled from localStorage state and included at the top of every API call's system prompt. This pattern avoids the need for agents to query the app state — they receive it directly.

Dual-Model Strategy

- **Anthropic API (Claude Sonnet):** All reasoning, classification, conversation, insights generation, and voice query interpretation. Claude excels at structured extraction, nuanced classification, and natural conversation.
- **Google Gemini API:** Vision/OCR only. Receipt scanning, document photo processing, handwriting recognition. Gemini's multimodal capabilities handle image-to-text better for these specific use cases.

This split keeps costs reasonable and uses each model where it's strongest. The Gemini integration is isolated in the AIScanModal component and doesn't affect the rest of the AI architecture.

6. Voice API & External Access

Vision

Family OS should answer questions wherever they're asked. A parent cooking dinner asks Alexa: "What homework does Maya have tonight?" A kid asks Siri: "What chores do I have left?" The answer comes from Family OS's data, interpreted by Claude, and delivered as natural speech.

Architecture (When Backend Is Added)

The voice API is a thin REST layer that accepts natural language queries and returns spoken responses:

Endpoint	Method	Description
/api/ask	POST	Natural language query → Claude interprets → queries data → returns spoken answer
/api/calendar	GET	Structured calendar data for a date range
/api/homework	GET	Assignments filtered by child, due date, status
/api/chores	GET	Chores filtered by assignee, status
/api/todos	GET	To-do items filtered by list, assignee
/api/goals	GET	Goal progress for individual or family

The /api/ask endpoint is the primary voice interface. It receives a text query (transcribed by Alexa/Siri), sends it to Claude along with the family's data context, and returns a natural language response suitable for text-to-speech. The structured endpoints exist for direct integrations that don't need AI interpretation.

Client-Side Preparation (Do Now)

Even without a backend, the app architecture should prepare for voice access by:

- **Centralizing data access:** Create a `FamilyDataService` class that abstracts all data reads. Today it reads from `localStorage`; tomorrow it reads from an API. All modules and agents use this service instead of accessing `localStorage` directly.
- **Query function:** Build an `askFamilyOS(question: string): Promise<string>` function that takes a natural language question, assembles the family context, calls Claude, and returns an answer. This function works identically in the browser and (future) on the server.
- **Structured data helpers:** Build helper functions like `getHomeworkDueTonight()`, `getTodaysEvents()`, `getPendingChores(childName)` that return clean, typed data. These become the API endpoints later.

Voice Platform Integration (Future)

- **Alexa:** Build an Alexa Skill using the Alexa Skills Kit. The skill's Lambda function calls `/api/ask` with the user's transcribed utterance. Requires an AWS account and Alexa developer account.
- **Siri:** Build Siri Shortcuts that call the REST API. Each shortcut maps to a common query: "What's on the family calendar today?" → `GET /api/calendar?date=today`. Requires an iPhone/Mac with Shortcuts app.
- **Google Home:** Build a Google Action using Dialogflow or the Actions SDK. Similar pattern to Alexa.

7. Interactive Walkthrough System

Concept

After conversational onboarding gathers the family's data, the user still doesn't know where anything is. The interactive walkthrough bridges this gap by guiding the user through each enabled module with contextual, step-by-step tooltip highlights.

This is not a video (which is static and gets outdated). It is a live, in-app guided tour that highlights real UI elements, explains what they do, and uses the family's actual data in examples: "Here's where you'll see Maya's homework — I already added the assignments I found in your email."

Implementation Approach

- **Library:** Shepherd.js or a custom lightweight tooltip system built with React portals and CSS transitions. Shepherd.js is preferred for its step-based tour API and scroll-to-element support.
- **Tour definition:** Each module exports an optional `tourSteps` array that defines the steps for that module's walkthrough. Steps include: target element selector, tooltip text (with template variables for family data), position, and optional action (e.g., "click this button").
- **AI narration:** Optionally, the walkthrough can call Claude to generate personalized narration for each step based on the family's profile and what data has already been imported.
- **Trigger:** The walkthrough launches automatically after onboarding completes (first-time) and is re-accessible from Settings or a "Take a tour" button on the Dashboard.

Tour Flow

The walkthrough adapts to enabled modules. A family with only Schoolwork and Calendar enabled sees a shorter tour than one with all modules active. The flow follows this sequence:

- Dashboard overview: quick stats, today's timeline, module widgets
- Each enabled module page: key features, how to add/edit items, what's auto-populated
- Settings page: how to enable more modules, configure integrations
- AI features: how the email scanner works, how to trigger a scan, where imported items appear
- Wrap-up: "You're all set! Here's a few things to try first." with 2–3 suggested actions

8. Data Model

Entity Relationship Overview

All entities belong to a Family. Users belong to a Family with a Role (Parent or Child). Students are the academic identity of child Users. Modules operate on their own entity types but share the family context.

Entity	Module	Key Fields	Storage Key
Family	Core	id, name, inviteCode	family_os_profile
User	Core	id, name, email, role, familyId	family_os_profile
Student	Schoolwork	id, familyId, name, grade, school	family_os_students
Assignment	Schoolwork	id, studentId, subject, title, dueDate, status, source	family_os_assignments
Chore	Chores	id, assigneeId, title, frequency, dueDate, status	family_os_chores
CalendarEvent	Calendar	id, title, start, end, location, provider	family_os_events
Transaction	Finance	id, date, description, amount, category, type	family_os_transactions
BudgetCategory	Finance	id, name, limit, spent, color	family_os_budgets
SavingsGoal	Finance	id, name, targetAmount, currentAmount, dueDate	family_os_savings
TodoItem	To-Dos	id, listId, title, assignee, dueDate, priority, done	family_os_todos
TodoList	To-Dos	id, name, familyId, color	family_os_todo_lists
Goal	Goals	id, title, owner, type, target, current, deadline	family_os_goals
Habit	Habits	id, title, frequency, assignee, streak, history[]	family_os_habits
ActionItem	Email Intel	id, title, childName, deadline, urgency, status	family_os_action_items
BehaviorUpdate	Email Intel	id, childName, type, details, points, date	family_os_behavior
Announcement	Email Intel	id, title, summary, source, date	family_os_announcements
ModuleConfig	Core	moduleId, enabled, settings	family_os_modules
EmailScanConfig	Email Intel	enabled, schoolDomains, includeClassDojo, lastScan	family_os_email_config

Cross-Module Relationships

Several modules have data relationships that enable richer Insights:

- **Chores ↔ Finance:** A completed chore can trigger allowance credit (Transaction with type: income, linked to child).
- **Goals ↔ Habits:** A behavioral goal spawns a Habit. Habit completion data feeds goal progress calculation.
- **Goals ↔ Finance:** A savings goal's progress is computed from Transactions in the matching category.
- **Email Intel ↔ Calendar/Schoolwork:** Classified emails create CalendarEvents and Assignments automatically.
- **Habits ↔ Insights:** Habit streaks and completion rates are a primary data source for the Family Flow Report.
- **All ↔ Voice API:** Every entity type is queryable via the askFamilyOS() function.

9. Integration Strategy

Integration via MCP (Model Context Protocol)

MCP is the preferred integration pattern for Family OS. Rather than building brittle one-off API integrations, each external service is accessed through an MCP server that the AI agent can call as a tool. This means the agent reasons about which tool to use, when, and how to combine data from multiple sources.

Integration	MCP Server	Status	Data Flow
Gmail	gmail.mcp.claude.com/mcp	Active	Read-only: search + read school emails
Google Calendar	gcal.mcp.claude.com/mcp	Planned	Read/write: sync events bidirectionally
Todoist	Community MCP server	Future	Read/write: sync to-do items
Apple Reminders	Via Siri Shortcuts	Future	Write: create reminders from to-dos
Google Classroom	Via Gmail notifications	Active	Read-only: parse notification emails
ClassDojo	Via Gmail notifications	Active	Read-only: parse notification emails
Bank Feeds	Plaid MCP or similar	Future	Read-only: import transactions

ClassDojo Integration (Email-Based)

ClassDojo does not provide a public API. Family OS integrates with ClassDojo by parsing the notification emails it sends to parents. This is a deliberate architectural choice — it’s more resilient than scraping a private API and doesn’t violate ClassDojo’s terms of service.

ClassDojo email types and how they’re classified:

- **Teacher messages:** Private messages from the teacher. Classified as `action_required` (if asking for something) or `announcement` (if informational).
- **Class Story posts:** Photos and updates from the classroom. Classified as `announcement`. Includes media links.
- **Behavior feedback:** Points earned or deducted. Classified as `behavior_update` with positive/negative type and point count.
- **Event reminders:** Scheduled class or school events. Classified as `calendar_event` with date/time extracted.
- **Portfolio approvals:** New student portfolio entries. Classified as `announcement`.

ClassDojo marketing and product promo emails (e.g., “Try ClassDojo Plus!”) are classified as irrelevant and discarded.

Email-as-Integration-Layer Pattern

The insight behind the email intelligence pipeline is that most school communication platforms send email notifications. By building a robust email parser, Family OS gets integration with ClassDojo, Google Classroom, Remind, Seesaw, and direct school emails all through a single pipeline. The classification agent is trained to recognize notification templates from each platform and extract the real content from within the email chrome.

10. Development Phases & Roadmap

Phase 1: Foundation (COMPLETE)

- Conversational AI onboarding via Anthropic API
- Dynamic family profile replacing hardcoded seed data
- localStorage persistence for all state
- DM Sans typography, warm onboarding UX

STATUS

Phase 1 is complete and working.

Phase 2: Email Intelligence (COMPLETE)

- Gmail MCP integration via Anthropic API
- Email scanning, classification, and routing pipeline
- ClassDojo notification parsing
- Dashboard: Needs Attention section + Recent from School feed
- Settings: Email Intelligence configuration panel
- Onboarding: Gmail connection step

STATUS

Phase 2 is complete and working.

Phase 3: Modular Architecture (NEXT)

- Implement module registry and ModuleConfig system
- Refactor App.tsx to dynamically register/render modules
- Sidebar, Dashboard, Settings all driven by module config
- Onboarding agent recommends modules based on family profile
- Disabled modules hidden from navigation, routes redirect to Dashboard

PRIORITY

This is the foundational refactor that enables everything else. Do this before adding new modules.

Phase 4: New Modules

- Shared To-Do Lists module (including multiple named lists)
- Goals module (individual + family goals, linked to other modules)
- Habit Tracker module (daily/weekly tracking, streaks, visualizations)
- Dynamic Insights page powered by Anthropic API with full family context

Phase 5: Interactive Walkthrough

- Shepherd.js or custom tooltip tour system
- Module-aware: tour adapts to enabled features
- AI-narrated steps using family's actual data
- Re-accessible from Settings and Dashboard

Phase 6: Voice API & External Access

- Introduce backend (Node.js + Express/Hono)
- Migrate from localStorage to database (SQLite/Turso or Supabase)
- REST API endpoints for all data types
- /api/ask endpoint with Claude for natural language queries
- Alexa Skill, Siri Shortcuts, Google Home Action
- FamilyDataService abstraction layer

Phase 7: Heartbeats & Automations

- Scheduled cron jobs for email scanning, daily digests
- Pre-built heartbeat templates with simple toggle UI
- Push notifications (web push or email)
- Custom automation rules (if X then Y)

Phase 8: Extended Integrations

- Google Calendar bidirectional sync via MCP
- Todoist / Apple Reminders sync for to-dos
- Bank feed integration via Plaid for Finance module
- Multi-family / multi-device sync

End of Architecture Document